

Java 8+ Features: Lambda, Stream API, Optional, Functional Interfaces

1. Lambda Expression

A **lambda expression** is a concise way to represent a function-like behavior that can be passed around as a value.

Before Java 8:

```
List<String> names = Arrays.asList("A", "B", "C");

for (String name : names) {
    System.out.println(name);
}
```

Lambda:

```
names.forEach(name -> System.out.println(name));
```

Syntax:

```
(parameters) -> expression
```

or:

```
(parameters) -> {
    statements;
}
```

Examples:

```
x -> x * 2
```

```
(a, b) -> a + b
```

```
() -> System.out.println("Hello")
```

2. Why Lambda?

Lambda helps reduce boilerplate when you need to pass behavior.

For example:

```
List<Payment> payments = ...;

payments.forEach(
    payment -> System.out.println(payment.getStatus())
)
```

```
};
```

Instead of creating a separate anonymous class.

3. Lambda Requires a Functional Interface

You cannot arbitrarily assign a lambda to any class.

This works:

```
@FunctionalInterface
interface PaymentProcessor {
    void process(Payment payment);
}
```

Then:

```
PaymentProcessor processor =
    payment -> System.out.println(
        payment.getAmount()
    );
```

Why?

Because `PaymentProcessor` has exactly **one abstract method**.

That brings us to functional interfaces.

4. Functional Interface

A **functional interface** is an interface having exactly **one abstract method**.

It can have:

- default methods
- static methods
- `Object` methods don't count toward the single-abstract-method restriction

Example:

```
@FunctionalInterface
public interface PaymentValidator {

    boolean validate(Payment payment);
}
```

Then:

```
PaymentValidator validator =
    payment -> payment.getAmount().signum() > 0;
```

The `@FunctionalInterface` annotation is not mandatory, but it tells the compiler to verify that the interface remains functional.

5. Built-in Functional Interfaces

You should know these very well.

Predicate<T>

Takes an input and returns `boolean`.

```
Predicate<Payment> successful =  
    p -> "SUCCESS".equals(p.getStatus());
```

Use:

```
successful.test(payment);
```

Think:

| **Condition / filtering**

Function<T, R>

Takes `T`, returns `R`.

```
Function<Payment, String> getStatus =  
    Payment::getStatus;
```

Think:

| **Convert one thing into another**

Consumer<T>

Takes input, returns nothing.

```
Consumer<Payment> logger =  
    p -> System.out.println(p.getTransactionId());
```

Think:

| **Consume/use the value**

Supplier<T>

Takes no input, returns a value.

```
Supplier<String> transactionIdSupplier =  
    () -> UUID.randomUUID().toString();
```

Think:

6. Easy Memory Trick

Predicate
 $T \rightarrow \text{boolean}$

Function
 $T \rightarrow R$

Consumer
 $T \rightarrow \text{void}$

Supplier
 $() \rightarrow T$

Memorize this.

7. Method References

Instead of:

```
payments.forEach(  
    p -> System.out.println(p)  
);
```

you can write:

```
payments.forEach(System.out::println);
```

This is called a **method reference**.

Common forms:

```
ClassName::staticMethod  
object::instanceMethod  
ClassName::instanceMethod  
ClassName::new
```

Example:

```
Function<String, Integer> parser =  
    Integer::parseInt;
```

Constructor:

```
Supplier<Payment> supplier =  
    Payment::new;
```

8. Stream API

The Stream API provides a declarative way to process collections of data.

Example:

```
List<Payment> payments = ...;

List<Payment> successfulPayments =
    payments.stream()
             .filter(p -> "SUCCESS".equals(p.getStatus()))
             .toList();
```

Conceptually:

```
Collection
  ↓
stream()
  ↓
filter
  ↓
map
  ↓
collect
```

A stream is **not a data structure**. It is a pipeline for processing data.

9. Stream Pipeline

A stream generally has:

```
Source
  ↓
Intermediate operations
  ↓
Terminal operation
```

Example:

```
payments.stream()
         .filter(p -> p.getAmount().signum() > 0)
         .map(Payment::getTransactionId)
         .toList();
```

Source

```
payments.stream()
```

Intermediate

```
filter(...)
map(...)
```

Terminal

```
toList()
```

10. Intermediate vs Terminal Operations

Intermediate

Return another Stream.

Examples:

```
filter  
map  
flatMap  
sorted  
distinct  
limit  
skip
```

They are generally **lazy**.

Terminal

Produce a result or side effect and terminate the stream.

Examples:

```
collect  
toList  
forEach  
count  
reduce  
findFirst  
findAny  
anyMatch  
allMatch  
noneMatch
```

11. filter()

Used to keep elements satisfying a condition.

```
List<Payment> successful =  
    payments.stream()  
        .filter(p -> "SUCCESS".equals(p.getStatus()))  
        .toList();
```

Think:

Which elements should remain?

12. map()

Transforms each element.

```
List<String> transactionIds =
```

```
payments.stream()
    .map(Payment::getTransactionId)
    .toList();
```

Input:

```
Payment
Payment
Payment
```

Output:

```
String
String
String
```

Think:

Transform A → B

13. filter() vs map()

Very common interview question.

filter

```
.filter(p -> p.getAmount().compareTo(BigDecimal.valueOf(1000)) > 0)
```

Keeps/removes elements.

map

```
.map(Payment::getAmount)
```

Transforms elements.

```
filter → Payment → Payment
map    → Payment → Amount
```

14. flatMap()

This is one of the more important 3-year Stream questions.

Suppose:

```
List<Customer> customers;
```

Each customer has:

```
List<Payment>
```

Now you want **all payments across all customers**.

Without `flatMap`, you'd end up with nested collections:

```
List<List<Payment>>
```

With:

```
List<Payment> payments =  
    customers.stream()  
        .flatMap(c -> c.getPayments().stream())  
        .toList();
```

Conceptually:

```
Customer 1 → [P1, P2]  
Customer 2 → [P3, P4]  
  
flatMap  
  
→ [P1, P2, P3, P4]
```

Think:

`map` can produce nested structures; `flatMap` flattens them.

15. `sorted()`

```
List<Payment> sorted =  
    payments.stream()  
        .sorted(Comparator.comparing(Payment::getAmount))  
        .toList();
```

Descending:

```
.sorted(  
    Comparator.comparing(Payment::getAmount).reversed()  
)
```

16. `distinct()`

Removes duplicates according to the elements' equality semantics.

```
List<String> statuses =  
    payments.stream()  
        .map(Payment::getStatus)  
        .distinct()  
        .toList();
```

If using custom objects, remember that equality semantics matter.

17. limit() and skip()

```
payments.stream()
    .skip(10)
    .limit(20)
    .toList();
```

Useful for selecting a portion of the stream, though database-side pagination is preferable when dealing with large database result sets.

18. findFirst() and findAny()

```
Optional<Payment> payment =
    payments.stream()
        .filter(p -> "SUCCESS".equals(p.getStatus()))
        .findFirst();
```

Why Optional?

Because there might be **no matching payment**.

19. anyMatch, allMatch, noneMatch

```
boolean exists =
    payments.stream()
        .anyMatch(p -> p.getAmount().compareTo(
            BigDecimal.valueOf(100000)) > 0);
```

anyMatch

At least one?

allMatch

Does every element satisfy the condition?

noneMatch

Does no element satisfy the condition?

These are useful because they can avoid processing the entire stream once the result is known.

20. reduce()

Used to combine elements into a single result.

Example: total payment amount.

```
BigDecimal total =
```

```
payments.stream()
    .map(Payment::getAmount)
    .reduce(
        BigDecimal.ZERO,
        BigDecimal::add
    );
```

Conceptually:

```
100 + 200 + 300
    ↓
    600
```

21. collect()

Commonly used to collect stream results.

```
List<Payment> successful =
    payments.stream()
        .filter(...)
        .collect(Collectors.toList());
```

Modern Java can also use:

```
.toList()
```

Be aware that `Stream.toList()` and `Collectors.toList()` do not promise exactly the same mutability characteristics. Don't blindly assume the returned list is mutable when using `toList()`.

22. Grouping

Very useful in real applications.

Suppose you want payments grouped by status:

```
Map<String, List<Payment>> grouped =
    payments.stream()
        .collect(
            Collectors.groupingBy(
                Payment::getStatus
            )
        );
```

Result conceptually:

```
SUCCESS → [P1, P2]
FAILED   → [P3, P4]
PENDING  → [P5]
```

This is a common interview exercise.

23. Counting With `groupBy`

```
Map<String, Long> countByStatus =
    payments.stream()
        .collect(
            Collectors.groupingBy(
                Payment::getStatus,
                Collectors.counting()
            )
        );
```

Result:

```
SUCCESS → 10
FAILED   → 3
PENDING  → 5
```

24. Optional

`Optional<T>` represents a value that **may or may not be present**.

Instead of:

```
Payment payment =
    repository.findById(id);

if (payment != null) {
    ...
}
```

you can have:

```
Optional<Payment> payment =
    repository.findById(id);
```

This makes absence explicit in the API.

25. Creating Optional

```
Optional<String> a =
    Optional.of("hello");
```

`of()` requires a non-null value.

This can throw `NullPointerException` if passed null.

`ofNullable()`

```
Optional<String> b =
    Optional.ofNullable(value);
```

Can represent:

```
value  
or  
empty
```

empty()

```
Optional<String> empty =  
    Optional.empty();
```

26. **isPresent()** / **isEmpty()**

```
if (payment.isPresent()) {  
    ...  
}
```

Modern style:

```
if (payment.isEmpty()) {  
    ...  
}
```

But don't treat `Optional` merely as a replacement for every `if` statement. Its strongest use is in expressing potentially absent return values.

27. **orElse()**

```
String status =  
    payment.map(Payment::getStatus)  
            .orElse("UNKNOWN");
```

If value exists:

```
SUCCESS
```

otherwise:

```
UNKNOWN
```

28. **orElse()** vs **orElseGet()**

Very common interview question.

orElse()

The fallback expression is evaluated eagerly.

```
payment.orElse(createDefaultPayment());
```

`createDefaultPayment()` is evaluated even when `payment` is present.

orElseGet()

The supplier is evaluated only when needed.

```
payment.orElseGet(this::createDefaultPayment);
```

Conceptually:

```
orElse  
→ fallback evaluated anyway  
  
orElseGet  
→ fallback evaluated only if empty
```

This matters when fallback creation is expensive or has side effects.

29. **orElseThrow()**

Very useful in Spring applications:

```
Payment payment =  
    repository.findById(id)  
        .orElseThrow(  
            () -> new PaymentNotFoundException(  
                "Payment not found: " + id  
            )  
        );
```

This connects directly to your previous **exception-handling** topic.

30. **map()** with Optional

Suppose:

```
Optional<Payment> payment;
```

You need the transaction ID:

```
Optional<String> id =  
    payment.map(Payment::getTransactionId);
```

The mapping only happens when a Payment exists.

31. Optional Anti-Patterns

Don't automatically use:

```
Optional<String> name;
```

as every entity field or DTO field.

Also avoid:

```
Optional<Payment> payment = ...;

if (payment.isPresent()) {
    Payment p = payment.get();
    ...
}
```

everywhere, because you're often recreating traditional null checks.

Prefer operations such as:

```
payment.map(...)
        .orElse(...)
```

or:

```
payment.ifPresent(...)
```

where appropriate.

A key rule:

`Optional` is generally most useful as a return type to express possible absence, rather than as a universal replacement for `null`.

32. Java 8 Interface Enhancements

Java 8 introduced **default and static methods in interfaces**.

Example:

```
interface PaymentProcessor {

    void process();

    default void validate() {
        System.out.println("Common validation");
    }

    static void log() {
        System.out.println("Payment");
    }
}
```

This allowed interfaces to evolve without forcing every existing implementation to provide every newly added method.

33. Default Method vs Abstract Method

```
void process();
```

Implementation is required.

Whereas:

```
default void validate() {  
    ...  
}
```

provides an implementation.

This can also come up when discussing Java 8 features.

34. Stream vs Collection

Another common question.

Collection

Stores data.

```
List  
Set  
Map
```

Stream

Processes data.

```
filter  
map  
sort  
reduce  
collect
```

A stream generally doesn't store the elements independently.

Example:

```
payments.stream()
```

creates a processing pipeline over the source collection.

35. Lazy Evaluation

Intermediate Stream operations aren't normally executed immediately.

Example:

```
Stream<Payment> stream =  
    payments.stream()  
        .filter(p -> {
```

```
        System.out.println("filter");
        return "SUCCESS".equals(p.getStatus());
    });
```

Nothing meaningful happens just from creating the pipeline.

After:

```
stream.toList();
```

the pipeline executes.

Conceptually:

```
stream()
  ↓
filter()   → lazy
map()      → lazy
  ↓
toList()   → triggers execution
```

36. Streams Don't Modify the Original Collection

For example:

```
List<Integer> numbers =
    Arrays.asList(1, 2, 3);

List<Integer> result =
    numbers.stream()
        .map(n -> n * 2)
        .toList();
```

The original:

```
[1, 2, 3]
```

The result:

```
[2, 4, 6]
```

Streams generally encourage transformation rather than modifying the source.

37. Sequential vs Parallel Stream

Normal:

```
payments.stream()
```

Parallel:

```
payments.parallelStream()
```

Parallel streams can split work across threads.

But:

Do not use `parallelStream()` simply because data is large.

It can introduce:

- thread-management overhead
- contention
- ordering issues
- difficult debugging
- poor performance for I/O
- interaction with shared mutable state

For payment/business processing, blindly using `parallelStream()` can be particularly dangerous.

38. Side Effects in Streams

Avoid code like:

```
List<Payment> result = new ArrayList<>();

payments.parallelStream()
    .forEach(result::add);
```

because `ArrayList` isn't thread-safe.

Even with a sequential stream, excessive mutation makes code harder to reason about.

Prefer:

```
List<Payment> result =
    payments.stream()
        .filter(...)
        .toList();
```

39. A Real BPS Example

Suppose you retrieve transactions:

```
List<Payment> payments = paymentRepository.findById(customerId);
```

You want transaction IDs for successful payments above ₹1,000:

```
List<String> transactionIds =
    payments.stream()
        .filter(p -> "SUCCESS".equals(p.getStatus()))
        .filter(p ->
            p.getAmount().compareTo(
                BigDecimal.valueOf(1000)
            ) > 0
        )
```

```
)  
.map(Payment::getTransactionId)  
.toList();
```

Pipeline:

```
Payments  
↓  
SUCCESS?  
↓  
Amount > 1000?  
↓  
Extract transactionId  
↓  
List<String>
```

This is exactly the sort of practical example you should be able to produce in an interview.

40. Another Real Example: Find Payment

```
Payment payment =  
    payments.stream()  
        .filter(p ->  
            transactionId.equals(  
                p.getTransactionId()  
            )  
        )  
        .findFirst()  
        .orElseThrow(() ->  
            new PaymentNotFoundException(  
                "Payment not found"  
            )  
        );
```

This connects:

```
Stream API  
+  
Optional  
+  
Custom Exception
```

That's a good 3-year-level example.

41. Common Java 8 Interview Questions

Lambda

1. What is a lambda expression?
2. What problem does lambda solve?
3. What is the syntax of lambda?
4. Can lambda work without a functional interface?
5. What is a method reference?

Functional interfaces

6. What is a functional interface?
7. What is `@FunctionalInterface`?
8. Predicate vs Function?
9. Consumer vs Supplier?
10. What functional interface is used by `filter()`?
11. What functional interface is used by `map()`?

Remember:

```
filter → Predicate  
map    → Function  
forEach → Consumer
```

42. Stream Interview Questions

12. What is Stream API?
13. Collection vs Stream?
14. Intermediate vs terminal operations?
15. Why are streams lazy?
16. `map()` vs `flatMap()`?
17. `filter()` vs `map()`?
18. `findFirst()` vs `findAny()`?
19. `reduce()`?
20. `collect()`?
21. `groupingBy()`?
22. `distinct()`?
23. `sorted()`?
24. Sequential vs parallel stream?
25. Can a stream be reused?

Important:

```
Stream<String> stream = ...;  
  
stream.count();  
stream.toList(); // IllegalStateException
```

A Stream is generally **single-use**.

43. Optional Interview Questions

26. Why use Optional?
27. `of()` vs `ofNullable()`?
28. `orElse()` vs `orElseGet()`?
29. `orElseThrow()`?
30. Should Optional be used as an entity field?
31. Can Optional contain null?

Answer:

An `Optional` itself should not contain null. `Optional.of(null)` throws `NullPointerException`; `Optional.ofNullable(null)` produces `Optional.empty()`.

44. Tricky Questions

Q: Which functional interface does `filter()` use?

```
Predicate<T>
```

because it returns boolean.

Q: Which one does `map()` use?

```
Function<T,R>
```

because it transforms one type into another.

Q: Which one does `forEach()` use?

```
Consumer<T>
```

because it consumes the value and returns nothing.

Q: Can you modify a collection while streaming over it?

Generally avoid structural modification of the source during stream traversal; it can cause `ConcurrentModificationException` or undefined/unreliable behavior depending on the source.

Q: Is Stream faster than a traditional loop?

Not inherently.

Streams improve readability/declarative processing and can enable certain optimizations, but a normal loop can be faster in some workloads.

Never claim:

“Stream API is always faster.”

45. Most Important for Your 3-Year Interview

Prioritize these:

Must know

```
Lambda  
Functional Interface  
Predicate / Function / Consumer / Supplier  
Stream  
filter
```

```
map
flatMap
sorted
distinct
collect
groupingBy
reduce
Optional
orElse
orElseGet
orElseThrow
```

Then know

```
Lazy evaluation
Method references
Parallel streams
Side effects
Stream reuse
Default methods
```

46. One-page memory map

```
JAVA 8+
|
├─ Lambda
│   └─ behavior as value
|
├─ Functional Interface
│   └─ one abstract method
|
│   └─ Predicate<T>
│       └─ T → boolean
|
│   └─ Function<T,R>
│       └─ T → R
|
│   └─ Consumer<T>
│       └─ T → void
|
│   └─ Supplier<T>
│       └─ () → T
|
├─ Stream API
│   └─ filter
│   └─ map
│   └─ flatMap
│   └─ sorted
│   └─ distinct
│   └─ limit
│   └─ collect
│   └─ reduce
│   └─ findFirst
|
└─ Optional
    └─ of
    └─ ofNullable
    └─ map
    └─ orElse
    └─ orElseGet
    └─ orElseThrow
```

Interview-ready explanation

“Java 8 introduced functional programming features such as lambda expressions, functional interfaces, the Stream API and Optional. A lambda provides a concise way to pass behavior, and it works with a functional interface containing one abstract method. Common functional interfaces are Predicate, Function, Consumer and Supplier. The Stream API provides declarative processing using operations such as filter, map, flatMap, collect and reduce, with intermediate operations being lazy until a terminal operation executes the pipeline. Optional represents a potentially absent value and helps make absence explicit; common operations include map, orElse, orElseGet and orElseThrow. In backend applications, these features are useful for collection processing, DTO transformation, filtering and safely handling repository results.”

For your **BPS interview**, make sure you can actually write from memory: **filter + map + collect**, **groupingBy**, **findFirst + orElseThrow**, and explain **orElse vs orElseGet**. Those are high-probability follow-ups.